

Sistema Multimodal de Recuperación y Búsqueda

Proyecto 2 — Base de Datos II · Ciclo 2026-1

Universidad de Ingeniería y Tecnología (UTEC)

Elmer Villegas · Juan Carlos Ticlia · Joseph Anderson Cose · Josué Hernández Yataco · Paulo Miranda

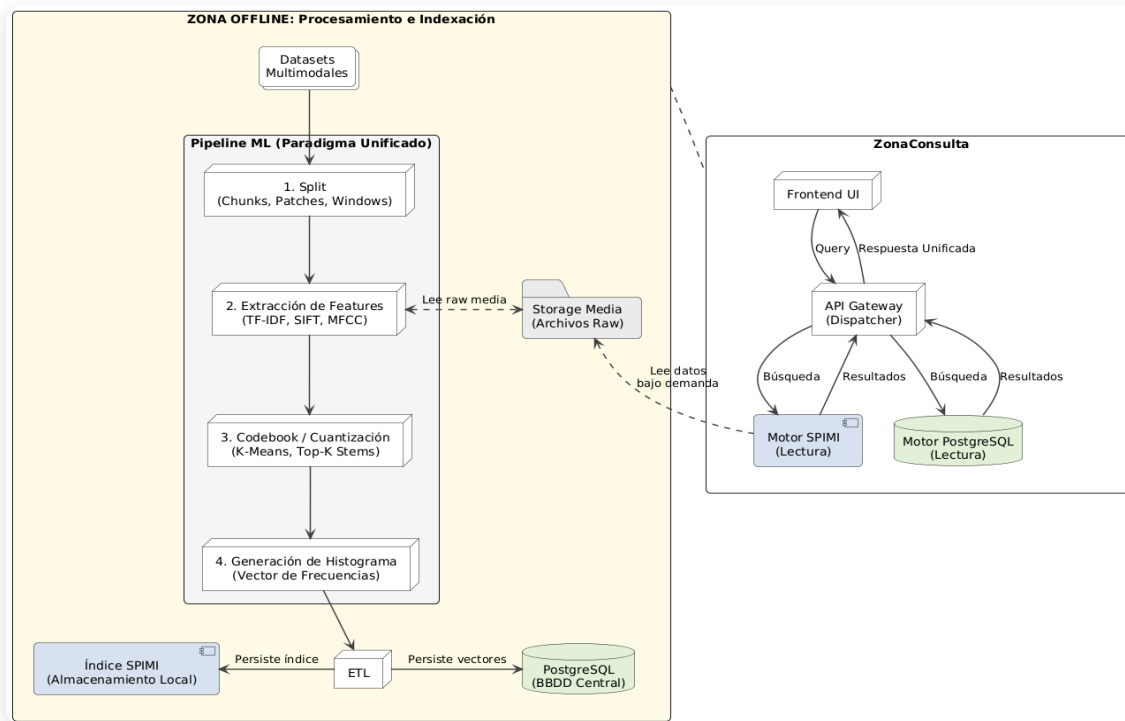
1. Arquitectura Unificada Multimodal

Un solo paradigma para texto, audio e imagen

El sistema aplica **la misma tubería de cuatro pasos** sin importar la modalidad, para poder comparar motores de forma justa sobre el mismo corpus: fragmentar el contenido, extraer sus características, reducirlas a un vocabulario finito, y construir un índice sobre ese vocabulario.

- **Fragmentar** — párrafos (texto), ventanas de tiempo (audio), regiones de la imagen
- **Extraer características** — un descriptor numérico distinto según la modalidad
- **Vocabulario finito (codebook)** — se comparte entre el motor propio y los nativos de Postgres
- **Índice invertido** — mismo motor (SPIMI) para las tres modalidades

Vista global del sistema



Zona offline — procesamiento

- Los datasets entran al pipeline ML común (Split → Extracción → Codebook → Histograma)
- El ETL persiste el índice SPIMI local y los vectores/tsvector en PostgreSQL

Zona de consulta — online

- El frontend envía la query al **API Gateway**, que decide a qué motor enrutarla
- El gateway consulta en paralelo al **motor SPIMI** y/o a **PostgreSQL**
- Ambos devuelven el mismo formato de respuesta unificada

Pipeline unificado end-to-end



Cinco fases, la **misma estructura para las tres modalidades** — solo cambia el extractor y el tamaño del codebook (**k**) en cada fase.

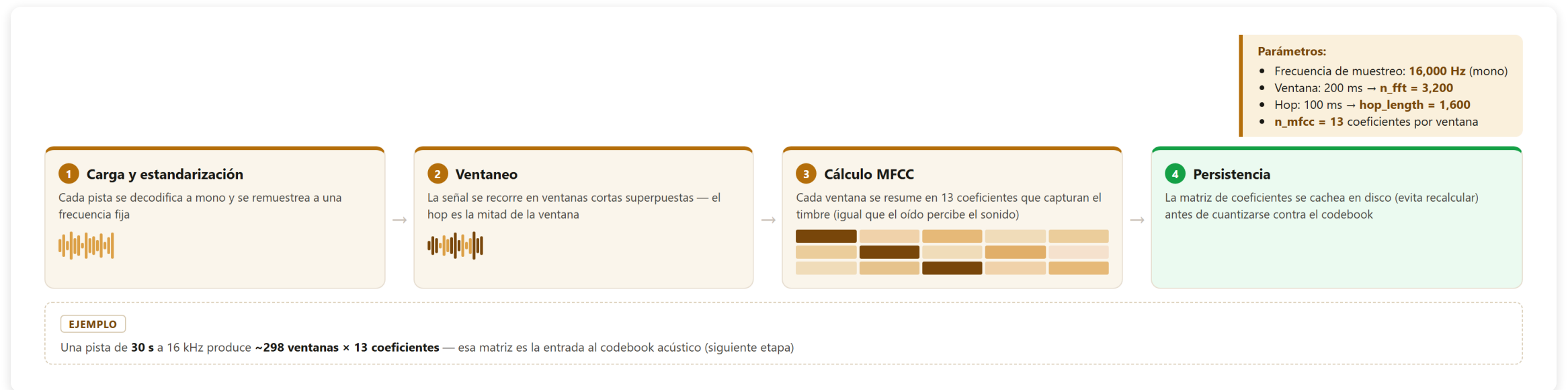
2. Extracción de Características por Modalidad

Texto — Normalización y reducción a raíces



Seis pasos, del texto crudo al histograma persistido — la misma lógica de fragmentar, extraer y cuantizar que ya vimos a nivel general, aplicada a texto.

Audio — Extracción de coeficientes cepstrales (MFCC)



El resultado se cachea por pista, ya que el cálculo de MFCC es la etapa más costosa del pipeline de audio.

Imagen — Detección de puntos clave (SIFT)



Esto permite reconocer la misma prenda o instancia visual aunque cambie el tamaño, el ángulo o la posición en la foto.

3. Construcción del Codebook (Diccionario)

Visual Words / Acoustic Words

La idea central: convertir descriptores locales continuos en un vocabulario **discreto y finito**, igual que un diccionario de palabras para texto.



Para texto el mismo principio se aplica sin K-Means: el vocabulario son las palabras más frecuentes del corpus (por la ley de Zipf, un puñado de raíces cubre la gran mayoría de las ocurrencias).

K-Means propio

Implementado desde cero para este proyecto — no usa librerías externas de clustering.

Modalidad	Tamaño del codebook	Muestra de entrenamiento
Audio (MFCC)	500 palabras acústicas	50,000 descriptores
Imagen (SIFT)	1,024 palabras visuales	150,000 descriptores

- Se entrena una sola vez por modalidad, sobre una muestra representativa del dataset completo
- Un centroide que se queda sin puntos asignados conserva su posición anterior, en vez de reiniciarse al azar — evita que el entrenamiento oscile
- Una vez entrenado, cada descriptor nuevo se asigna al centroide más cercano para construir el histograma del documento

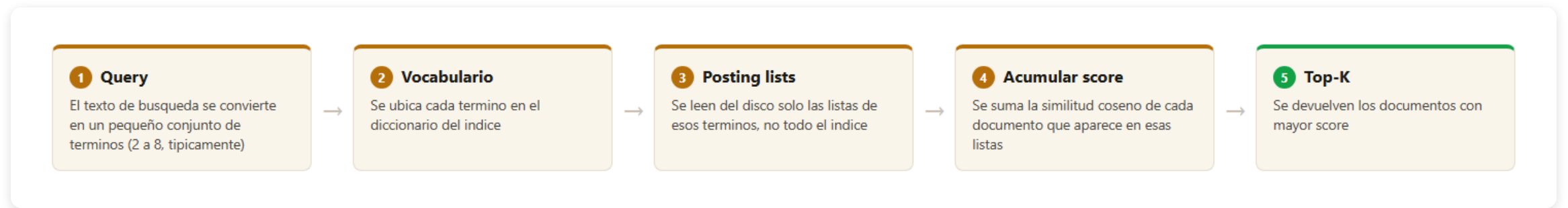
4. Implementación del Índice Invertido (SPIMI)

SPIMI — Single-Pass In-Memory Indexing

Resuelve indexar un corpus **más grande que la RAM disponible**, sin necesitar tenerlo todo en memoria a la vez.



Consulta sobre el índice invertido



Solo se leen del disco las posting lists de los términos de la query — nunca el vocabulario completo. El mismo mecanismo sirve para texto, audio e imagen: un "codeword" visual o acústico se busca exactamente igual que una palabra.

5. Motores Nativos en PostgreSQL

Full-text search — GIN vs GiST

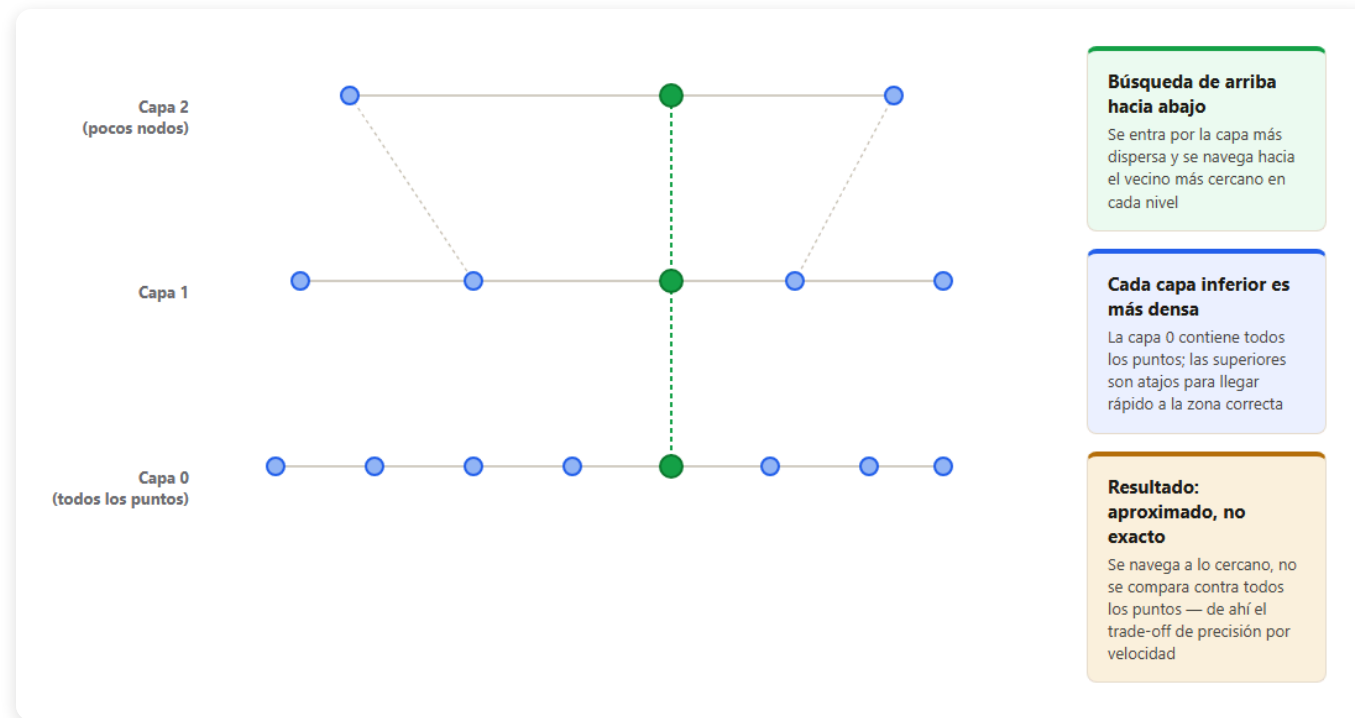
Postgres mantiene una columna de búsqueda de texto que se actualiza sola cada vez que cambia el contenido — no hace falta código adicional para mantenerla sincronizada.

Índice	Cómo busca	Mejor para
GIN	Índice invertido clásico: palabra → lista de documentos	Lectura muy rápida, escritura más costosa
GiST	Árbol balanceado con firmas aproximadas	Escritura más barata, lectura con una verificación extra

Ambos devuelven un score de relevancia (qué tan bien calza el texto con la consulta) y permiten ordenar los resultados por ese score.

Búsqueda vectorial — pgvector + HNSW

HNSW (Hierarchical Navigable Small World): grafo organizado en capas que permite encontrar vecinos cercanos sin comparar contra todo el dataset.



Postgres expone esto como un operador nativo de distancia sobre los mismos vectores del pipeline.

6. Datasets y Preprocesamiento

Fuentes de datos

Spotify Songs Lyrics

~40,000 letras, solo en inglés

FMA-(small, Large)

~40,000 pistas de audio, 8 géneros

Fashion Product Images

~44,000 productos con imagen + descripción

App	Modalidades	Dataset	Auth
App · Música	Texto + Audio	Spotify Songs Lyrics (Kaggle) + FMA-small	Kaggle token + HTTPS
App · Fashion	Texto + Imagen	Fashion Product Images (Kaggle)	Kaggle token

- **Letras:** se conservan solo las que están en inglés, para que el vocabulario de stopwords y el stemmer sean consistentes en todo el corpus
- **Fashion:** la descripción de cada producto combina su nombre con los atributos estructurados del catálogo (categoría, color, uso, temporada, etc.)

Preprocesamiento y muestreo

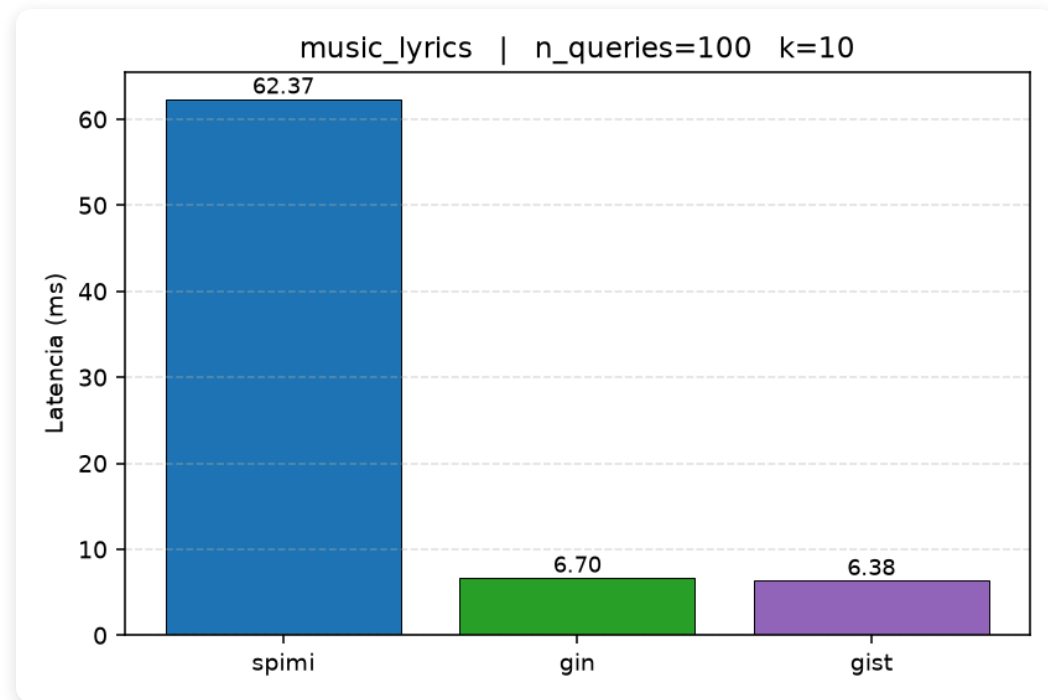
- **Metadatos:** cada pista de audio se enriquece con su título, artista y género antes de retornarse como respuesta al frontend
- **Pareo de modalidades:** en Fashion, cada producto tiene imagen y descripción, por lo que es un documento bimodal. En música, letras y audio vienen de fuentes distintas, así que cada canción es texto-solo o audio-solo, nunca ambas
- **Muestreo para K-Means:** en vez de usar todos los descriptores del dataset (inviabile en memoria), se toma una muestra uniforme representativa — grande y aleatoria, pero reproducible

7. Evaluación Experimental y Resultados

Metodología

- **Cargas:** $N = 10,000 / 20,000 / 30,000 / 40,000$ documentos
- **Fuente de verdad:** la etiqueta de clase almacenada en Postgres — un resultado es TP si su etiqueta coincide con la de la query
 - `music/lyrics` y `music/audio` → columna `genre`
 - `fashion/description` y `fashion/image` → columna `subcategory`
- **Métricas:** latencia (avg / p50 / p95 ms), throughput (QPS), memoria pico (RSS), tamaño en disco del índice, accesos a disco en SPIMI (`io_seeks`)
- **recall_norm@10** = $TP / \min(K, \text{relevantes})$ — evita que un motor con AND estricto (GIN/GiST) parezca "perfecto" solo por devolver 1-2 hits
- **Consultas:** 100 por (`motor, modalidad, N`), las mismas para los 4 motores, $k=10$. Cada query es un item real del dataset y se excluye del top-K para no contarla como acierto trivial

Resultados — Texto: Letras (Music)

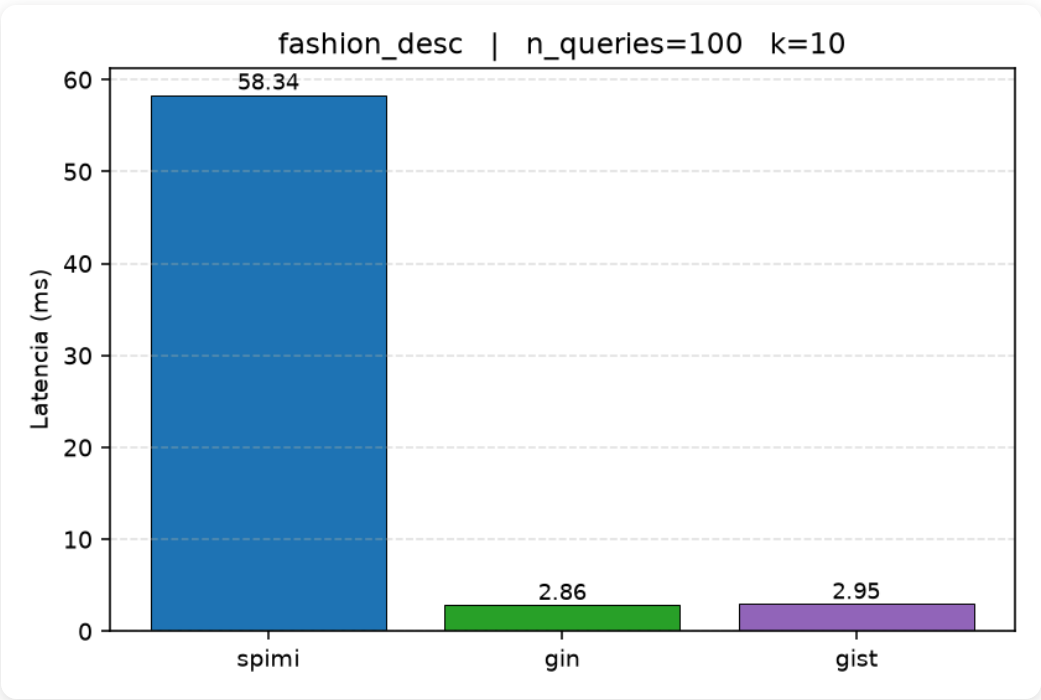


N = 40,000 · SPIMI GIN GiST

Motor	avg ms	QPS	recall@10	precision@10	recall_norm@10
SPIMI	82.7	12	0.000	0.014	0.014
GIN	8.5	117	0.001	0.033	0.013
GiST	7.7	130	0.001	0.033	0.013

Los tres motores **empatan** en precision y recall_norm — la etiqueta de género no correlaciona con similitud textual sobre TF-IDF de letras. **recall@10** está en el suelo por tope estructural (cientos de canciones por género, k = 10). La comparación real es por latencia: **GIN/GiST ~10× más rápidos**; SPIMI solo responde 78/100 queries.

Resultados — Texto: Descripción (Fashion)

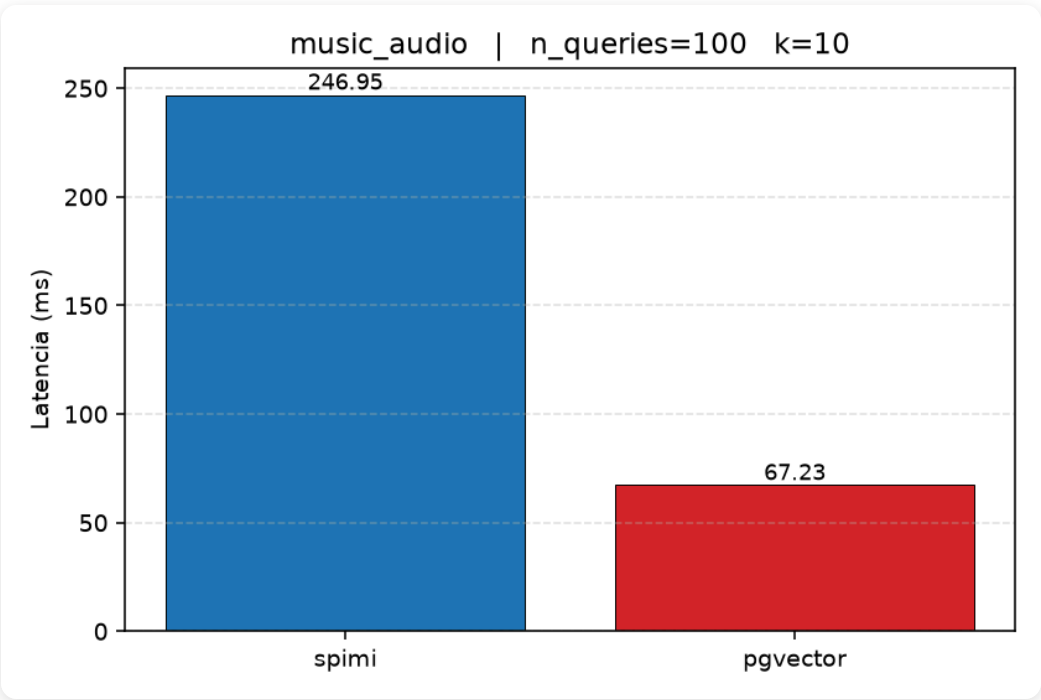


N = 40,000 · SPIMI GIN GiST

Motor	avg ms	QPS	recall@10	precision@10	recall_norm@10
SPIMI	76.2	13	0.009	0.988	0.892
GIN	3.2	314	0.003	1.000	0.514
GiST	3.0	331	0.003	1.000	0.514

GIN/GiST son **~24x más rápidos** pero solo responden 44/100 queries (AND estricto). Su **precision = 1.000** engaña — cuando responden con 1-2 hits, los aciertan, pero cubren mucho menos del top-10 posible. **recall_norm** normaliza por k y expone que SPIMI cubre ~90% vs ~51%. **recall@10** sigue bajo por tope estructural (cientos de items por subcategoría).

Resultados — Audio (Music)

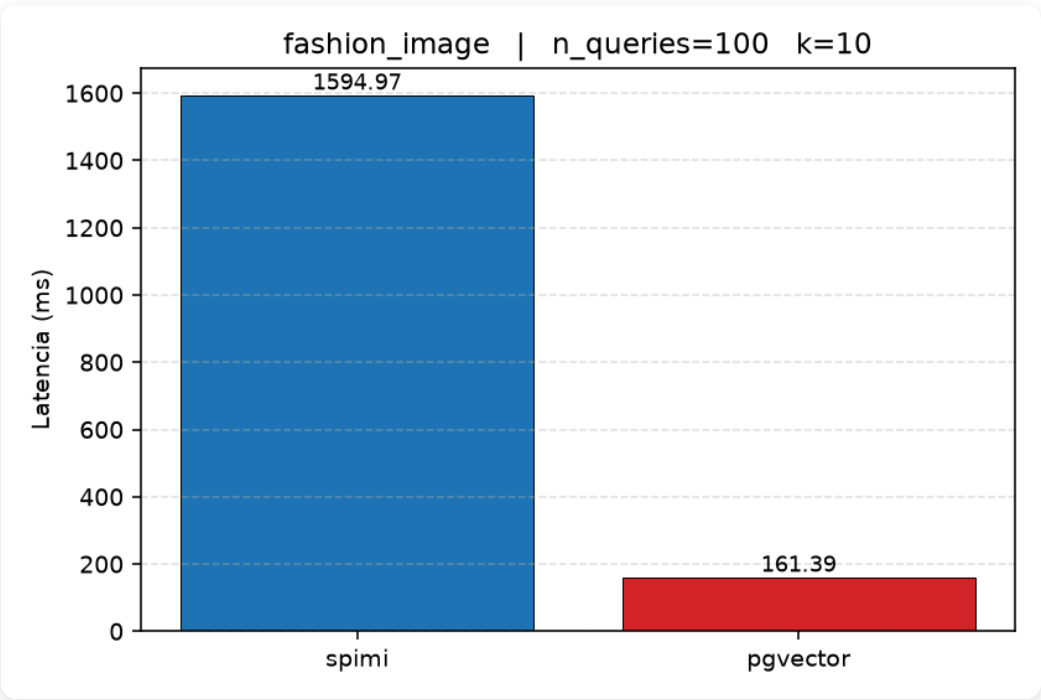


SPIMI **pgvector** (HNSW) · similitud coseno sobre codebook MFCC (k = 500)

N	Motor	avg ms	QPS	recall@10	precision@10	recall_norm@10
10k	SPIMI	138	7.2	0.006	0.380	0.342
10k	pgvector	79	12.7	0.005	0.343	0.309
40k	SPIMI	320	3.1	0.002	0.285	0.257
40k	pgvector	83	12.0	0.001	0.249	0.224

pgvector ~4× más rápido, pero **SPIMI le gana en precision y recall_norm** — el ranking exacto por coseno no pierde información, HNSW aproxima. **recall@10** bajo por tope de clase (miles de pistas por género vs k = 10). Brecha manejable en esta escala.

Resultados — Imagen (Fashion)

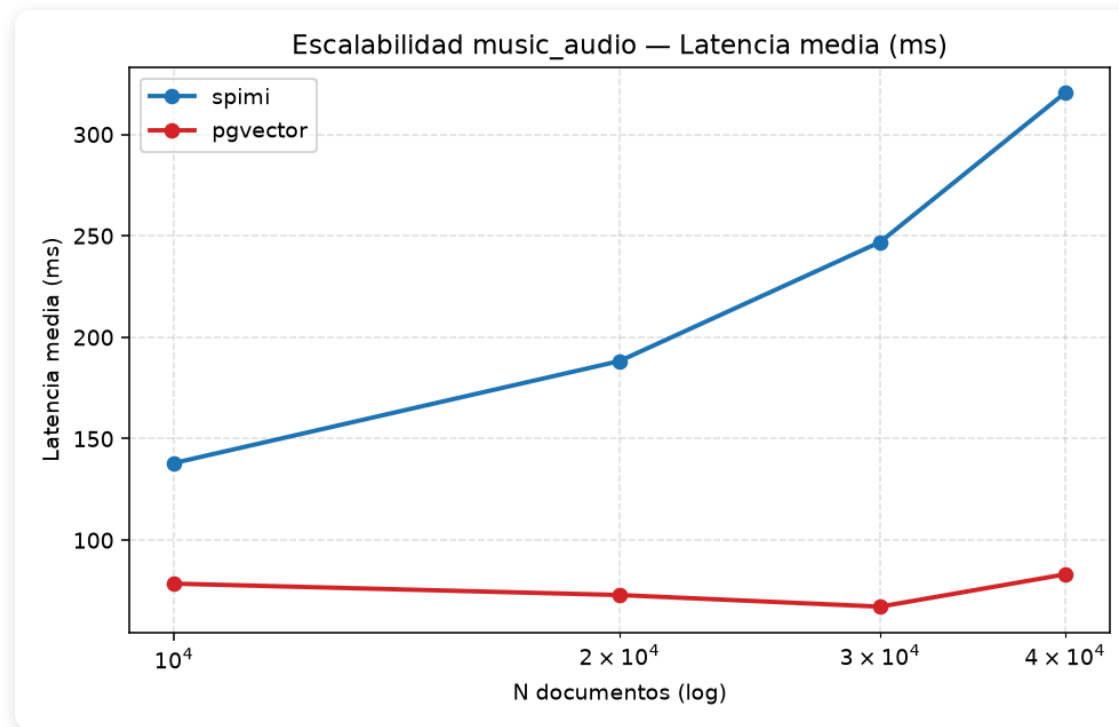
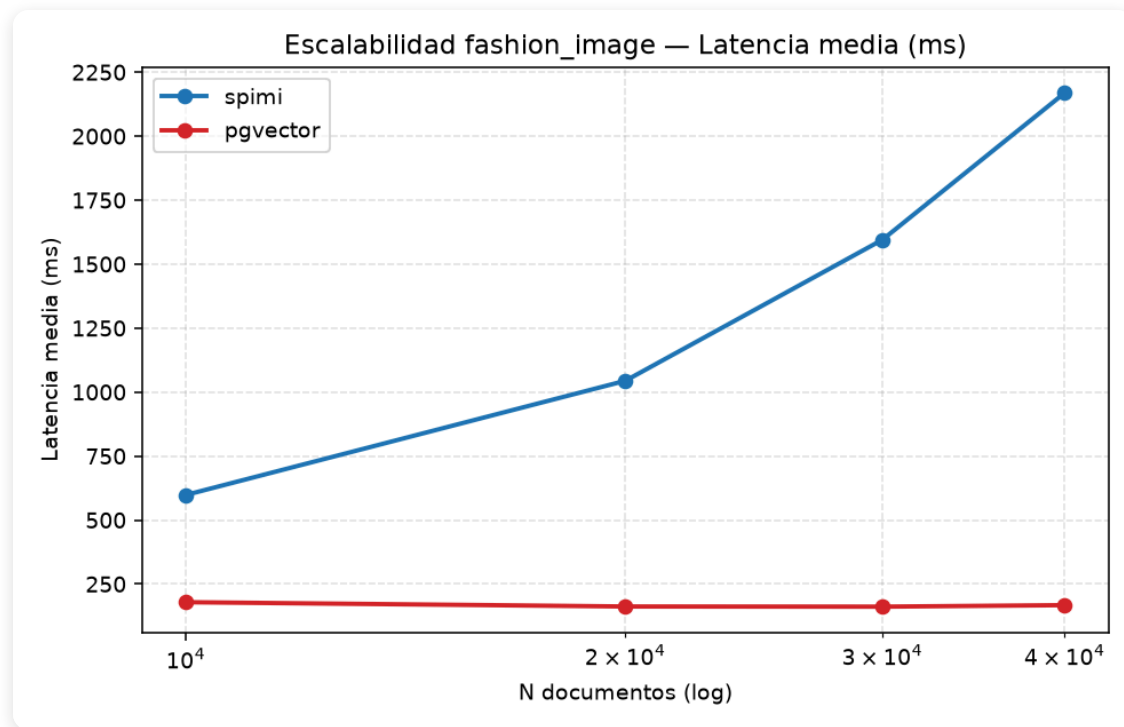


SPIMI **pgvector** (HNSW) · similitud coseno sobre codebook SIFT (k = 1,024)

N	Motor	avg ms	QPS	recall@10	precision@10	recall_norm@10
10k	SPIMI	598	1.7	0.006	0.540	0.486
10k	pgvector	179	5.6	0.005	0.560	0.505
40k	SPIMI	2 169	0.5	0.002	0.396	0.356
40k	pgvector	167	6.0	0.002	0.473	0.426

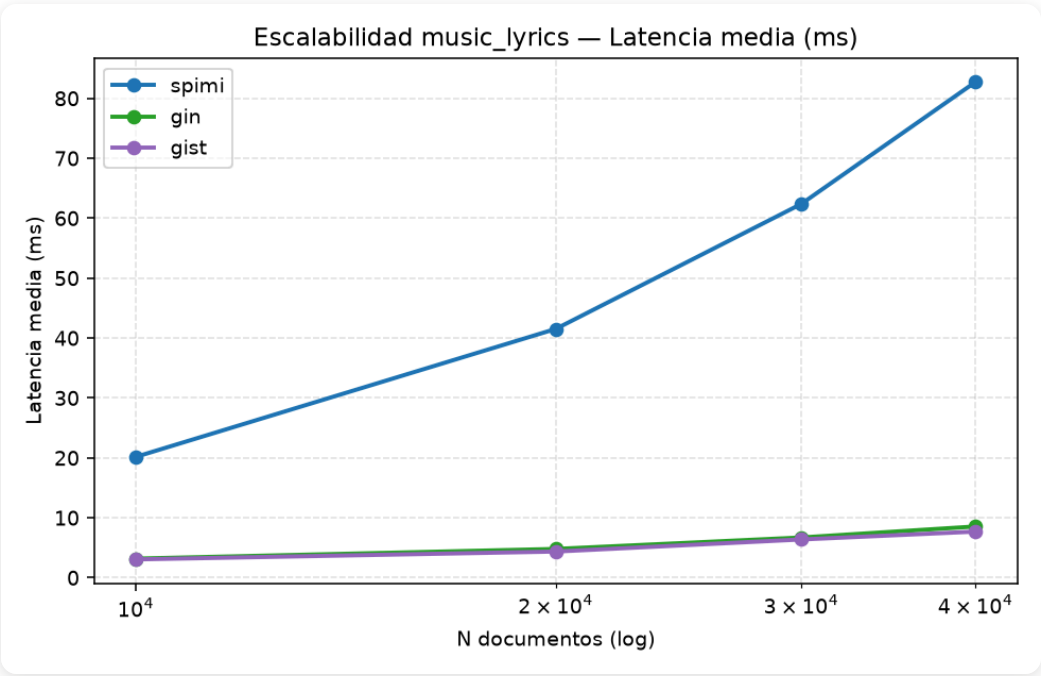
pgvector **gana en los tres frentes**: ~13× más rápido, mejor precision y mejor recall_norm. SPIMI cruza los 2 s por query a N=40k (fuera de rango interactivo).
recall@10 bajo por tope estructural (cientos de productos por subcategoría).

Escalabilidad — latencia vs N (10k → 40k)



En **imagen** SPIMI pasa de 598 ms → 2 168 ms mientras pgvector se mantiene entre 167-179 ms. En **audio** SPIMI escala de 138 ms → 320 ms mientras pgvector oscila en ~79-83 ms. La pendiente cuenta la historia: **SPIMI crece con N** (scan lineal sobre posting lists sin skip lists), **HNSW es plano** (navegación logarítmica del grafo).

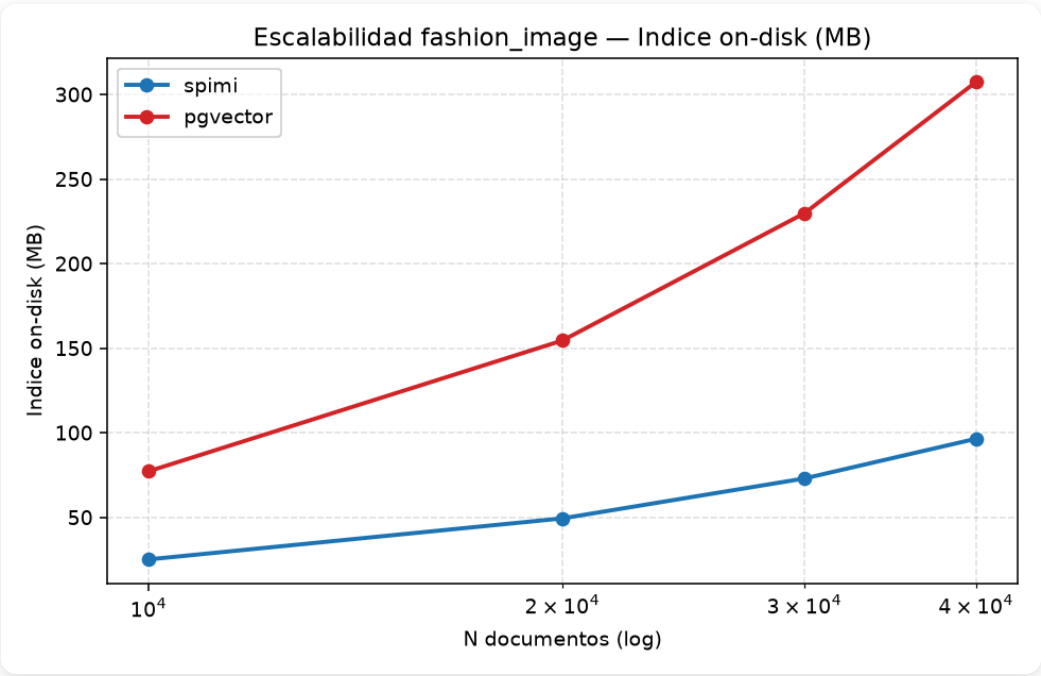
Escalabilidad — factor de crecimiento por modalidad



Modalidad	Motor	10k → 40k	Factor
lyrics	SPIMI	20 → 82 ms	4.1×
lyrics	GIN	3.2 → 8.5 ms	2.7×
desc	SPIMI	19 → 76 ms	4.0×
desc	GIN	1.9 → 3.2 ms	1.7×
audio	SPIMI	138 → 320 ms	2.3×
audio	pgvector	79 → 83 ms	1.06×
image	SPIMI	598 → 2 168 ms	3.6×
image	pgvector	179 → 167 ms	0.94×

SPIMI escala casi lineal en texto (4× datos → 4× latencia). **pgvector es plano** — con más datos la brecha se agranda. GIN también crece pero con base tan baja (~3 ms) que es irrelevante.

Costo en disco y RAM (N = 40,000)



Motor	Modalidad	RSS pico (MB)	Índice on-disk (MB)
SPIMI	lyrics	396	132.5
SPIMI	desc	505	9.6
SPIMI	audio	497	27.6
SPIMI	image	642	96.4
GIN	lyrics	396	16.5
GIN	desc	505	3.0
pgvector	audio	497	102.9
pgvector	image	646	307.3

- **SPIMI en texto ~8× más pesado que GIN** — postings crudas como JSON, sin compresión de posting trees
- **SPIMI en audio/imagen 3-4× más liviano que pgvector** — histogramas BoVW son sparse (50-200 dim no-cero) vs. embeddings densos
- **pgvector paga overhead del grafo HNSW** — 307 MB para 40k × 1024 float

Impacto de dimensionalidad y accesos a disco

Modalidad	Codebook k	Descriptores por doc	SPIMI @ N=40k	Motor nativo @ N=40k	Ratio
Texto (lyrics)	1 000	~50-200 stems únicos	82.7 ms	GIN 8.5 ms	10×
Texto (desc)	1 000	~10-50 stems únicos	76.2 ms	GIN 3.2 ms	24×
Audio (MFCC)	500	~300 frames	320.4 ms	pgvector 83.2 ms	4×
Imagen (SIFT)	1 024	~500-2 000 keypoints	2 168.9 ms	pgvector 167.2 ms	13×

- **Codebook chico + descriptores densos = SPIMI se degrada.** En imagen cada codeword aparece en 40-60% de los documentos → posting lists enormes → scan TAAT costoso. En texto natural funciona: cada canción usa ~50-200 stems únicos de 1 000 (Zipf), las posting lists son manejables.
- **HNSW no escala con la dimensión del vector.** dim 500 → 1 024 solo duplica la latencia de pgvector (83 → 167 ms); en SPIMI la latencia se multiplica por **7×** (320 → 2 169 ms).
- **Accesos a disco (instrumentados en SPIMI).** El bottleneck real son los **io_seeks** por término, no los bytes totales leídos — solo se cargan las posting lists de los términos de la query, nunca el vocabulario completo.
- **Mitigaciones aplicadas.** Cuantización a k codewords, coseno + normalización L2, poda IDF de términos ubicuos, subsampling estratificado para K-Means, HNSW aproximado en lugar de KNN exacto.

8. Conclusiones

Conclusiones del proyecto

- **Arquitectura unificada** — el mismo pipeline de 4 pasos cubre texto, audio e imagen sin código específico por modalidad
- **Postgres nativo domina en velocidad** — a N=40k, 10× (lyrics), 24× (desc), 4× (audio), 13× (imagen) más rápido que SPIMI
- **SPIMI gana en precisión donde importa** — recall_norm superior en Descripción (0.892 vs 0.514) y Audio (0.257 vs 0.224) por ranking exacto
- **Recomendación:** texto → **GIN**, audio e imagen → **pgvector**, descripción → híbrido

Limitación principal. SIFT + BoVW captura textura, no "objeto" — retrieval semántico por categoría necesitaría features CNN (ResNet, CLIP), no más **k** en K-Means.

9. Aplicaciones Implementadas (Demo)

App · Búsqueda Musical Inteligente

Modalidad primaria: Audio + Texto

- Buscar canciones **por letra** (full-text: **SPIMI** **GIN** **GiST**)
- Buscar canciones **por similitud acústica** subiendo un clip de audio (**SPIMI** **pgvector**, sobre histogramas MFCC)
- Frontend **GRID**: modo comparación lado a lado de hasta 3-4 motores con la misma query, leaderboard de latencia/ranking, letra resaltada en las palabras que hicieron match, reproductor de audio propio para escuchar el resultado

App · Recomendación Multimodal (Fashion)

Modalidad primaria: Imagen + Descripción

- Buscar productos **por descripción** (texto: **SPIMI** **GIN** **GiST**)
- Buscar productos **visualmente similares** subiendo una foto (**SPIMI** **pgvector**, sobre histogramas SIFT)
- Cada resultado muestra miniatura, categoría, subcategoría y descripción completa del producto
- Mismo frontend GRID, misma identidad de color por motor, mismo modo de comparación

Gracias

Sistema Multimodal de Recuperación y Búsqueda

Proyecto 2 — Base de Datos II · UTEC